

Project Testing Specification

Project: **FitTrack AI** — Personal Fitness & Nutrition Tracking Platform

Testing is focused on the REST APIs implemented in the supplied repository. The project includes a comprehensive Postman collection with example requests, environment variables, and automated JWT token-variable handling.

1. Testing Tool

Postman is used to verify API endpoints, response HTTP status codes, user authentication workflows, request body validations, workout/nutrition CRUD operations, calorie & metric aggregations, and Gemini-powered AI recommendation generation.

2. Test Areas

Test Area	Validation Checks & Coverage
Registration	Valid registration; missing required fields; short/weak password handling; duplicate email rejection.
Login	Valid credentials; missing credentials; invalid email or password errors.
Profile	Valid JWT bearer token authorization; protected route checks for missing or expired tokens.
Fitness & Nutrition Logs	Create, list, update, and delete workout or meal entries; required parameter verification; duplicate entries; user data ownership protection.
Metrics Tracking	Valid log retrieval (calories, steps, macros); missing parameter handling; resource-not-found handling; fallback logic when third-party services are offline.
AI Workout Summary	Required input validation; authorized request flow; Gemini model generation and fallback handling.
AI Nutrition Insights	Dietary preference validation; authorized request flow; Gemini recommendation generation and fallback response testing.
General API	Base health route, unhandled/unknown route handling, standardized JSON structures, centralized error response format.

3. Postman Demonstration Sequence

- Register User:** Send a request to register a test user. The Postman script automatically stores the returned JWT into the `token` collection variable.
- Login:** Authenticate with the same credentials to verify token re-issuance and automatic header updating.

3. **Get Profile:** Fetch the authenticated user profile using the Bearer token.
4. **Create Log:** Add a new workout/meal entry and store the generated `log_id`.
5. **Fetch Logs:** Retrieve the full list of saved fitness and nutrition logs.
6. **Update Log:** Modify details of the created log entry using `log_id`.
7. **Get Progress Metrics:** Request progress metrics (calories, macros, volume) for a targeted timeframe.
8. **Generate AI Summary:** Request an AI-generated fitness summary using the bearer token.
9. **Generate AI Recommendation:** Request personalized AI nutrition recommendations.
10. **Delete Log:** Remove the workout/meal log entry to verify complete CRUD lifecycle.

4. Expected Results

- **200 / 201 OK** Successful requests return JSON with a explicit success indicator and response payload.
- **401 UNAUTHORIZED** Protected endpoints strictly reject requests lacking a valid Bearer token.
- **400 BAD REQUEST** Validation errors return clear field-level error messages.
- **404 NOT FOUND** Non-existent routes or missing resource IDs return standardized error objects.
- **RESILIENCY** Fitness metrics and AI recommendation services remain testable via built-in fallback behavior when external API keys are unavailable.

5. Testing Evidence

The project documentation references an API testing demonstration. For final submission, full-page screenshots of major Postman responses and the link to the final demonstration video are attached to the project repository.